

Trường Đại học Khoa học tự nhiên
Khoa Công nghệ thông tin



BÁO CÁO ĐỒ ÁN

Ares's Adventure

Cơ sở trí tuệ nhân tạo

Giáo viên hướng dẫn

Bùi Duy Đăng

Nguyễn Thanh Tình

Lê Nhựt Nam

Huỳnh Lâm Hải Đăng

Mục lục

1	Thông tin chung	3
1.1	Thông tin thành viên	3
1.2	Bảng phân chia công việc	3
1.3	Thông tin đồ án	4
1.4	Mô tả	4
2	Đánh Giá Kết Quả Đồ Án	4
3	Phân tích thuật toán	6
3.1	Thuật toán BFS	6
3.1.1	Giới thiệu	6
3.1.2	Thực thi	6
3.1.3	Diagram	6
3.2	Thuật toán DFS	6
3.2.1	Giới thiệu	6
3.2.2	Thực thi	6
3.2.3	Diagram	7
3.3	Thuật toán UCS	7
3.3.1	Giới thiệu	7
3.3.2	Thực thi	7
3.3.3	Diagram	7
3.4	Thuật toán Greedy Best First Search	8
3.4.1	Giới thiệu	8
3.4.2	Thực thi	8
3.4.3	Diagram	8
3.5	Thuật toán A*	9
3.5.1	Giới thiệu	9
3.5.2	Thực thi	9
3.5.3	Diagram	9
4	Hàm hỗ trợ	10
4.1	Tính toán Heuristic	10

4.1.1	Độ tin cậy của hàm heuristic	10
4.1.2	Lựa chọn phương pháp tính khoảng cách	10
4.1.3	Cách vận hành của hàm heuristic	11
4.2	Kiểm tra deadlock	11
5	Kết quả thực nghiệm	13
5.1	Kết thuật toán	13
5.1.1	Bản đồ 1	13
5.1.2	Bản đồ 2	14
5.1.3	Bản đồ 3	15
5.2	Biểu đồ	16
5.3	Phân tích test case	18
6	Kết Luận	18
6.1	Ưu điểm	18
6.1.1	BFS	18
6.1.2	DFS	19
6.1.3	UCS	19
6.1.4	A*	19
6.1.5	GBFS	19
6.2	Nhược điểm	19
6.2.1	BFS	19
6.2.2	DFS	19
6.2.3	UCS	20
6.2.4	A*	20
6.2.5	GBFS	20
7	Biến thể thuật toán A* (Weighted A*)	20
8	Tham khảo	21

1 Thông tin chung

1.1 Thông tin thành viên

MSSV	Họ và tên	Email
23127115	Mạch Quốc Tấn	mqtan23@clc.fitus.edu.vn
23127334	Nguyễn Thành Dâng	ntdang23@clc.fitus.edu.vn
23127344	Trương Thành Đạt	ttdat23@clc.fitus.edu.vn
23127082	Nguyễn Thị Khánh Linh	ntklinh23@clc.fitus.edu.vn

Bảng 1: Bảng thông tin thành viên

1.2 Bảng phân chia công việc

Công việc	Phụ trách	Trạng thái	%
Thực thi thuật toán BFS	Thành Dâng	Hoàn Thành	100%
Thực thi thuật toán DFS	Khánh Linh	Hoàn Thành	100%
Thực thi thuật toán UCS	Thành Đạt	Hoàn Thành	100%
Thực thi thuật toán A*	Quốc Tấn	Hoàn Thành	100%
Tạo file input với các mức độ khác nhau	Quốc Tấn Thành Dâng	Hoàn Thành	100%
Output và đồ họa	Khánh Linh Thành Đạt Quốc Tấn	Hoàn Thành	100%
Báo cáo	Thành Dâng	Hoàn Thành	100%
Video hướng dẫn chạy thuật toán	Khánh Linh	Hoàn Thành	100%
Thực hiện thuật toán bổ sung	Quốc Tấn Thành Dâng	Hoàn Thành	50%

Bảng 2: Bảng phân chia công việc

1.3 Thông tin đề án

- Bảng quản lý tiến độ làm việc: Excel Task
- Link video hướng dẫn sử dụng GUI: Video
- Source code: Github

1.4 Mô tả

Đề án Ares-Adventure là đề án giải bài toán tìm đường được phát triển bằng Python, tích hợp bốn thuật toán tìm kiếm: Tìm kiếm theo chiều rộng (**BFS**), Tìm kiếm theo chiều sâu (**DFS**), Tìm kiếm theo chi phí đồng nhất (**UCS**) và Tìm kiếm A* (**A* Search**), Tìm kiếm tham lam ưu tiên tốt nhất (**GBFS**). Mục tiêu so sánh hiệu quả của các thuật toán này trong việc tìm lời giải tối ưu cho các bài toán di chuyển, nơi ares cần phải di chuyển các viên đá vào các công tắc được chỉ định.

Để nâng cao trải nghiệm và giúp việc triển khai thuật toán trở nên dễ dàng hơn, Ares-Adventure được tích hợp giao diện đồ họa (**GUI**) sử dụng Pygame. Nhờ đó có thể dễ dàng hiểu cách thức hoạt động của từng thuật toán, phát hiện các điểm mạnh và hạn chế, cũng như tối ưu hóa chiến lược tìm kiếm trong các bài toán di chuyển và sắp xếp phức tạp.

2 Đánh Giá Kết Quả Đề Án

Sau quá trình thực hiện đề án và hoàn thiện sản phẩm, nhóm chúng em tự đánh giá kết quả làm việc của mình với số điểm **11/12**, dựa trên các tiêu chí sau:

- **Hoàn thành yêu cầu đề ra:**
 - Nhóm đã triển khai và thực thi đúng các thuật toán **BFS**, **DFS**, **UCS** và **A*** theo yêu cầu.
 - Kết quả thực hiện của các thuật toán được thể hiện rõ ràng trong báo cáo
 - Hoàn thành video demo
- **Xây dựng bộ dữ liệu đầu vào (Input) với nhiều cấp độ khác nhau:**
 - Các input được thiết kế theo từng mức độ khó khác nhau nhằm đánh giá hiệu suất của các thuật toán một cách chính xác và toàn diện.
 - Các input file được chia thành 3 mức độ khác nhau, mỗi mức độ sẽ khác nhau về số lượng viên đá, kích thước mê cung, như ở mức độ 1 sẽ có 1 viên đá, mức độ 2 sẽ có 2 viên đá, mức độ 3 sẽ có 3 viên đá, và cùng một mức độ sẽ có kích thước mê cung và sự bố trí vật cản khác nhau. Tạo điều kiện để phân tích thuật toán trên nhiều khía cạnh hơn.
- **Hiển thị kết quả trên hai phương diện: GUI và output file:**
 - **Output file:**
 - * Chương trình tích hợp khả năng xuất kết quả đường đi của nhân vật Ares vào file đầu ra.

- * Khi thực thi, người dùng có thể quan sát quá trình chinh phục mê cung của Ares và lưu lại các kết quả này.

– **Giao diện đồ họa (GUI):**

- * Các phần tử trong game (viên đá, công tắc, nhân vật Ares,...) đều do nhóm tự thiết kế, giúp tăng tính trực quan và sinh động.
- * Hệ thống âm thanh được lồng ghép trong quá trình thực thi, mô phỏng đường đi để tăng thêm tính hấp dẫn.
- * Giao diện thân thiện, dễ sử dụng, các thao tác chủ yếu thông qua phím bấm và có hướng dẫn trực quan ngay trên màn hình.

• **Video trình bày kết quả:**

- Nhóm đã thực hiện một video demo để trình bày chi tiết kết quả của đồ án, thể hiện quá trình thực thi các thuật toán và giao diện chương trình.

• **Phân tích và đánh giá các thuật toán:**

- Thực hiện phân tích, so sánh hiệu suất của các thuật toán trên nhiều bộ dữ liệu đầu vào khác nhau.
- Đánh giá các tiêu chí như số bước đi, chi phí, số lượng node đã duyệt, thời gian thực thi và bộ nhớ sử dụng.

• **Mở rộng với các thuật toán khác:**

- Ngoài các thuật toán yêu cầu, nhóm đã thử nghiệm thêm một số phương pháp mở rộng để so sánh và đánh giá hiệu quả tìm đường trong trò chơi.

Với những kết quả đạt được, nhóm chúng em tự tin rằng sản phẩm đã đáp ứng đầy đủ các yêu cầu đề ra và có tính ứng dụng cao.

3 Phân tích thuật toán

3.1 Thuật toán BFS

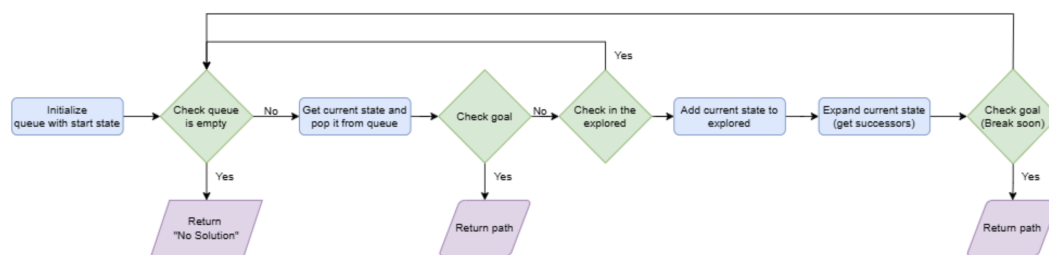
3.1.1 Giới thiệu

BFS (Breadth-First Search - Tìm kiếm theo chiều rộng) là một thuật toán duyệt đồ thị theo chiều rộng, bắt đầu từ một đỉnh và duyệt các đỉnh kề trước khi chuyển sang các đỉnh xa hơn. Thuật toán này được cài đặt bằng hàng đợi (queue) và hoạt động theo nguyên tắc FIFO (First In, First Out).

3.1.2 Thực thi

Áp dụng thuật toán vào bài toán Ares adventure ở đây BFS sẽ lưu tất cả trạng thái của đồ thị bao gồm vị trí đá, đích đến và vị trí hiện tại của ares vào queue sau đó lần lượt duyệt qua tất cả các trạng thái này. Thuật toán sẽ tiếp tục mở rộng cho đến khi trạng thái hiện tại tại tất cả các đá đều đã được đặt tại vị trí đích đến và tất cả trạng thái trùng lặp sẽ đảm bảo được bỏ qua tránh trường hợp lặp lại không cần thiết.

3.1.3 Diagram



Hình 1: Mô tả cách thức thực thi của BFS

3.2 Thuật toán DFS

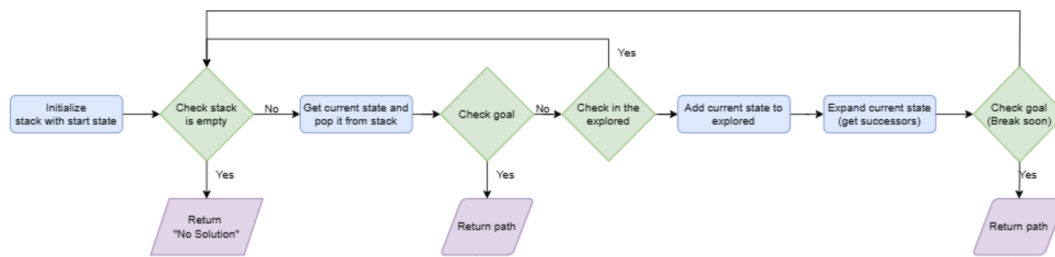
3.2.1 Giới thiệu

DFS (Depth-First Search - Tìm kiếm theo chiều sâu) là một thuật toán duyệt đồ thị theo chiều sâu, bắt đầu từ một đỉnh và đi càng xa càng tốt theo từng nhánh trước khi quay lại. DFS thường được triển khai bằng ngăn xếp (stack) theo nguyên tắc LIFO (Last In, First Out) hoặc sử dụng đệ quy.

3.2.2 Thực thi

Áp dụng thuật toán vào bài toán Ares Adventure, từ trạng thái ban đầu, bản đồ (bao gồm vị trí tường, đá, công tắc và Ares) có thể chuyển sang một số trạng thái khác có hướng di chuyển đúng quy định. Những trạng thái mới sẽ lần lượt được thêm vào stack, thuật toán sẽ tiếp tục mở rộng trạng thái bằng cách ưu tiên đi sâu vào một nhánh trước khi quay lại kiểm tra các hướng khác. Nghĩa là đối với DFS, thuật toán này sẽ đi theo một đường và đào sâu vào đường đó đến khi không thể đi tiếp thì chuyển hướng. DFS sẽ dừng lại khi tất cả các viên đá được đặt đúng vị trí. Để tránh lặp lại không cần thiết, các trạng thái đã duyệt sẽ được lưu lại để loại bỏ trùng lặp.

3.2.3 Diagram



Hình 2: Mô tả cách thức thực thi của DFS

3.3 Thuật toán UCS

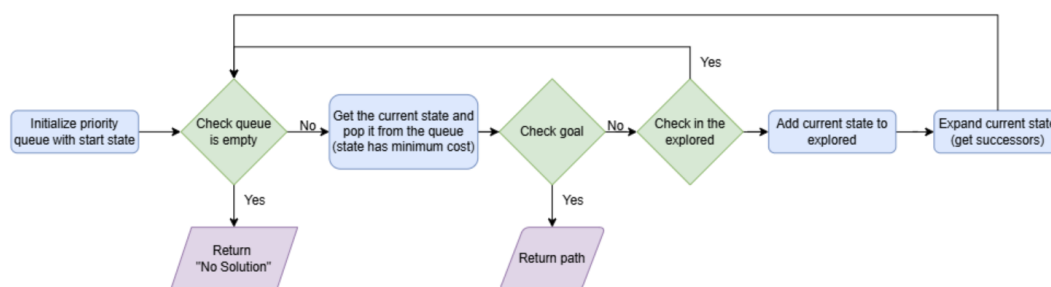
3.3.1 Giới thiệu

UCS (Uniform Cost Search) là một thuật toán tìm kiếm trong đồ thị, trong đó các trạng thái được duyệt theo chi phí, với mục tiêu tìm đường đi có chi phí thấp nhất từ điểm xuất phát đến đích. UCS sử dụng một hàng đợi ưu tiên để chọn trạng thái có chi phí thấp nhất và tiếp tục mở rộng các trạng thái kế tiếp.

3.3.2 Thực thi

Áp dụng thuật toán UCS vào bài toán Ares Adventure, thuật toán sẽ thực hiện các bước sau: Khởi tạo một hàng đợi ưu tiên với trạng thái bắt đầu và chi phí ban đầu là 0. Chọn trạng thái có chi phí thấp nhất trong hàng đợi. Mở rộng trạng thái đó, tính toán chi phí từ trạng thái hiện tại đến các trạng thái kế. Nếu trạng thái kế chưa được duyệt, cập nhật chi phí và thêm vào hàng đợi. Lặp lại quá trình trên cho đến khi đạt được trạng thái đích hoặc không còn trạng thái nào trong hàng đợi. Quá trình này đảm bảo rằng UCS sẽ tìm được con đường có chi phí thấp nhất từ điểm bắt đầu đến đích.

3.3.3 Diagram



Hình 3: Mô tả cách thức thực thi của UCS

3.4 Thuật toán Greedy Best First Search

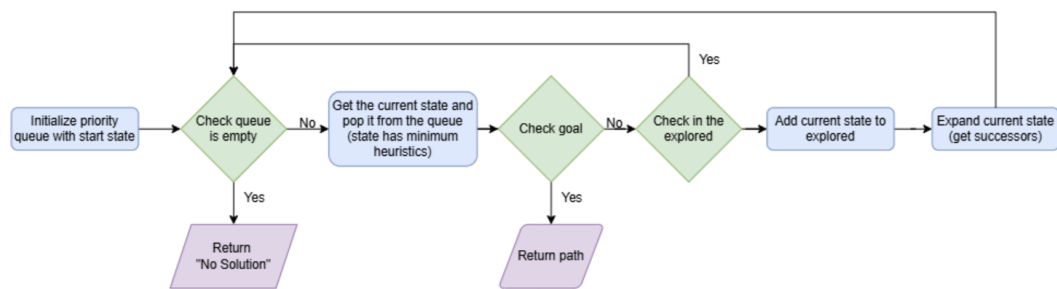
3.4.1 Giới thiệu

Greedy Best-First Search (GBFS) là một thuật toán tìm kiếm có thông tin, hoạt động dựa trên nguyên tắc ưu tiên mở rộng trạng thái có ước lượng tốt nhất trước. Thuật toán sử dụng hàm heuristic $h(n)$ để đánh giá khoảng cách ước lượng từ trạng thái hiện tại đến mục tiêu và duy trì một hàng đợi ưu tiên để luôn chọn trạng thái có giá trị $h(n)$ nhỏ nhất. Quá trình này lặp lại cho đến khi tìm thấy trạng thái đích

3.4.2 Thực thi

Trong bài toán Ares's Adventure, thuật toán Greedy Best-First Search (GBFS) giúp Ares tìm đường hiệu quả bằng cách ưu tiên di chuyển đến các vị trí có khoảng cách ước lượng gần mục tiêu nhất. Thay vì dựa vào khoảng cách thực tế đã đi, thuật toán sử dụng một hàm heuristic để đánh giá trạng thái và chọn hướng đi tốt nhất. Trong quá trình thực thi, GBFS sẽ đưa các trạng thái mới vào hàng đợi ưu tiên, với chỉ số ưu tiên là giá trị heuristic của trạng thái đó. Ares sẽ luôn chọn trạng thái có ước lượng gần mục tiêu nhất để mở rộng, ưu tiên vào việc tiếp cận các viên đá và tìm cách đẩy chúng vào switch một cách nhanh chóng. Điều này giúp Ares di chuyển có tốt hơn, tránh đi lòng vòng và tối đa hóa bài toán.

3.4.3 Diagram



Hình 4: Mô tả cách thức thực thi của GBFS

3.5 Thuật toán A*

3.5.1 Giới thiệu

Thuật toán A* là một thuật toán tìm kiếm có thông tin, kết hợp giữa chi phí thực tế đã đi và ước lượng chi phí còn lại để đưa ra quyết định mở rộng trạng thái. Mỗi trạng thái n sẽ được đánh giá bằng hàm:

$$f(n) = g(n) + h(n)$$

Trong đó:

- $g(n)$ là chi phí thực tế từ trạng thái ban đầu đến trạng thái hiện tại.
- $h(n)$ là giá trị heuristic, ước lượng khoảng cách từ trạng thái hiện tại đến mục tiêu.

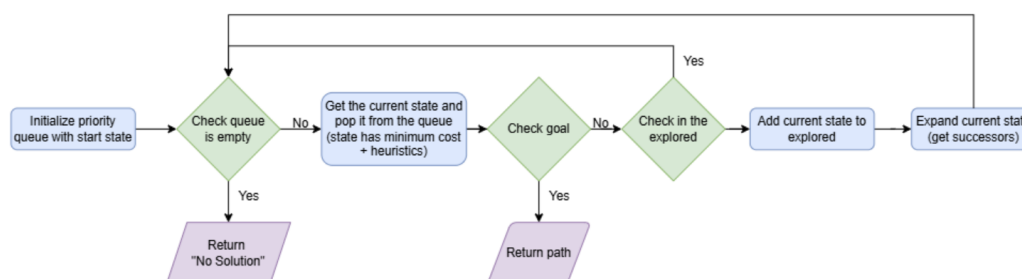
Việc tìm kiếm không chỉ dựa vào dự đoán của heuristic mà còn dựa vào chi phí thực tế giúp cho A* không chỉ tìm kiếm theo hướng có tối ưu nhất mà còn đảm bảo tìm ra đường đi có chi phí thấp nhất nếu hàm heuristic được thiết kế hợp lý. Điều này giúp thuật toán trở nên hiệu quả hơn so với các thuật toán tìm kiếm tham lam hay tìm kiếm theo chi phí đồng nhất.

Với trạng thái bắt đầu, sau đó phát triển ra các trạng thái khác, và bỏ các trạng thái được sinh ra cùng với trọng số f của trạng thái đó vào hàng đợi ưu tiên, thì khi lấy một trạng thái mới ra để phát triển, trạng thái đó sẽ có trọng số nhỏ nhất, hay nói cách khác là khoảng cách đi thực tế và ước lượng tới đích có thể là nhỏ nhất. Cứ tiếp tục như vậy, cho đến khi trạng thái đích được lấy ra từ hàng đợi, thì quãng đường đi thực tế sẽ là nhỏ nhất (bởi lúc đó ước lượng bằng 0 do đã ở đích, nên ta có trọng số lúc này chính là khoảng cách đi thực tế nhỏ nhất).

3.5.2 Thực thi

Trong bài toán "Hành Trình của Ares", thuật toán này giúp Ares tìm đường hiệu quả bằng cách đánh giá từng trạng thái dựa trên tổng chi phí thực tế đã đi được và ước lượng khoảng cách còn lại đến mục tiêu. Cụ thể, mỗi trạng thái mới sẽ được đưa vào hàng đợi ưu tiên với chỉ số đánh giá là tổng của chi phí thực tế từ điểm xuất phát đến trạng thái hiện tại và giá trị heuristic dự đoán khoảng cách đến đích. Trong quá trình thực thi, Ares sẽ luôn chọn trạng thái có tổng chi phí nhỏ nhất để mở rộng, giúp tối ưu hóa hành trình bằng cách di chuyển hợp lý, nhanh chóng tiếp cận các viên đá và tìm cách đẩy chúng vào switch mà không lãng phí bước đi. Nhờ khả năng đảm bảo tìm ra đường đi ngắn nhất, A* giúp Ares di chuyển hiệu quả, tránh những nước đi không cần thiết và tối ưu hóa quá trình giải quyết bài toán.

3.5.3 Diagram



Hình 5: Mô tả cách thức thực thi của A*

4 Hàm hỗ trợ

4.1 Tính toán Heuristic

Trong mê cung mà Ares đang đứng, những thông tin mà agent có thể biết được bao gồm:

- Vị trí của các switch.
- Vị trí của các bức tường.
- Vị trí của các viên đá.

Dựa vào các thông tin này, Ares có thể ước lượng chi phí đường đi còn lại để đến được mục tiêu. Chi phí này được tính toán dựa vào hàm heuristic.

4.1.1 Độ tin cậy của hàm heuristic

Một hàm heuristic đáng tin cậy cần thỏa mãn điều kiện:

$$h(x) \leq h^*(x)$$

trong đó:

- $h(x)$ là chi phí ước lượng từ trạng thái hiện tại đến mục tiêu.
- $h^*(x)$ là chi phí thực tế tối ưu.

Giả sử có n viên đá và n switch, tổng chi phí thực tế có thể được biểu diễn như sau:

$$h^*(x) = h_1^* + h_2^* + \dots + h_n^*$$

trong đó h_i^* là chi phí đẩy viên đá thứ i vào một switch bất kỳ. Khi xét đến chi phí di chuyển viên đá, nó phụ thuộc vào trọng lượng của viên đá nhân với quãng đường cần di chuyển.

4.1.2 Lựa chọn phương pháp tính khoảng cách

Có hai phương pháp phổ biến để ước lượng khoảng cách:

- **Khoảng cách Euclid:** Luôn nhỏ hơn khoảng cách thực tế vì bỏ qua vật cản như tường, có thể đi chéo. Do đó, tuy đáng tin cậy nhưng có thể không tối ưu.
- **Khoảng cách Manhattan:** Chỉ tính toán dựa trên di chuyển theo trục ngang và dọc, phù hợp với môi trường có vật cản, đảm bảo ước lượng chính xác hơn.

Trong bài toán này, khoảng cách Manhattan được chọn vì nó duy trì tính chấp nhận được của heuristic:

$$h(x) \leq h^*(x)$$

giúp thuật toán tìm kiếm hoạt động hiệu quả hơn.

4.1.3 Cách vận hành của hàm heuristic

Hàm heuristic trong bài toán này được tính như sau:

1. Tìm viên đá gần Ares nhất, tính khoảng cách từ Ares đến viên đá đó, gọi là `Ares_to_stone`.
2. Duyệt qua tất cả các viên đá, tìm switch gần nhất cho từng viên đá và tính khoảng cách.
3. Tổng chi phí heuristic được tính bằng:

$$h(x) = \text{Ares_to_stone} + \sum_{i=1}^n d_i$$

với d_i là khoảng cách từ viên đá i đến switch gần nhất của nó.

Heuristic này đảm bảo rằng $h(x) \leq h^*(x)$ vì:

- Không tính đến việc Ares di chuyển qua lại giữa các viên đá.
- Trong thực tế, rất ít khi mỗi viên đá có một switch riêng biệt, làm cho chi phí thực tế cao hơn.

Do đó, cách tiếp cận này là hợp lý và giúp tối ưu tốc độ tìm kiếm

4.2 Kiểm tra deadlock

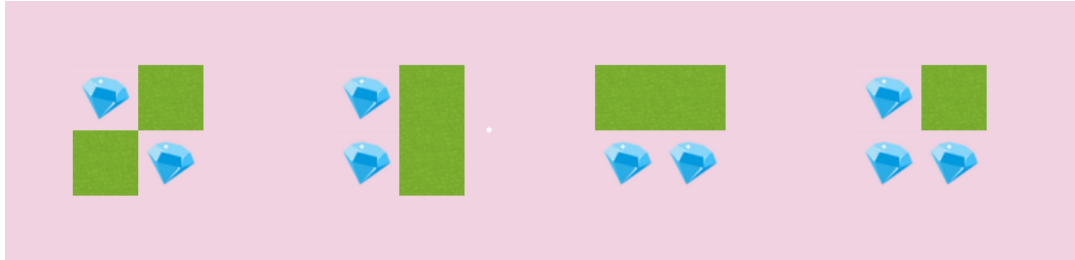
Trong quá trình phát triển các trạng thái, sẽ có những trạng thái dù có tiếp tục phát triển cũng không thể đạt đến trạng thái đích. Cụ thể, đó là các trạng thái mà tại đó các viên đá rơi vào vị trí không thể di chuyển được nữa, nhưng vị trí đó cũng không phải là switch cần đặt viên đá.

Các trạng thái như vậy gây tiêu hao bộ nhớ, làm mất thời gian xử lý những trạng thái không có giá trị, ảnh hưởng trực tiếp đến việc tìm kiếm lời giải. Chúng em gọi các trạng thái như vậy là **deadlock**, thuật ngữ thường được dùng để chỉ trạng thái tiến thoái lưỡng nan, không thể xử lý tiếp. Nếu có thể phát hiện và loại bỏ các trạng thái deadlock, bộ nhớ và thời gian xử lý sẽ được tối ưu hóa đáng kể. Vì vậy, chúng em xây dựng một hàm để kiểm tra xem trạng thái hiện tại có phải là deadlock hay không.

Cách kiểm tra trạng thái deadlock: Để kiểm tra một trạng thái có phải là deadlock hay không, chúng em dựa vào vị trí của các viên đá. Nếu viên đá đã ở đúng vị trí cần thiết (trên switch), thì dù không thể di chuyển, trạng thái đó vẫn hợp lệ. Tuy nhiên, nếu viên đá chưa ở vị trí đúng, cần kiểm tra xem viên đá có đang rơi vào trạng thái không thể di chuyển nữa hay không.

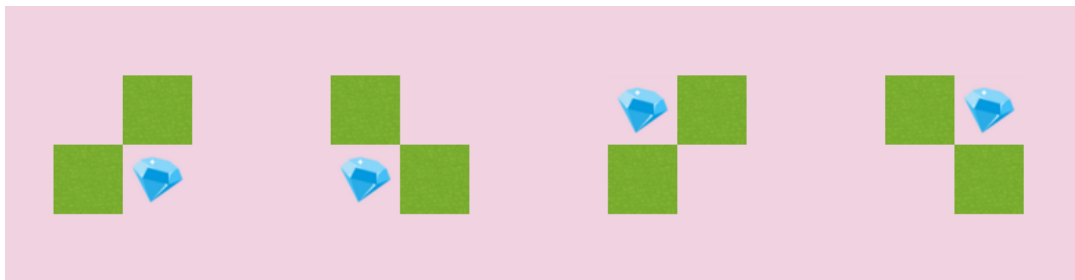
Hai trường hợp phổ biến dẫn đến deadlock:

1. **Deadlock dạng hình vuông:** Nếu một viên đá cùng với các vật thể xung quanh (như tường hoặc các viên đá khác) tạo thành một hình vuông, viên đá đó không thể di chuyển. Ví dụ, nếu góc trái trên của viên đá tiếp giáp với một vật thể là tường hoặc đá, và đồng thời phía trên và bên trái của viên đá đều là tường hoặc đá, thì trạng thái này là deadlock.



Hình 6: Trường hợp minh họa

2. **Deadlock dạng hai tường kề nhau:** Nếu một viên đá bị kẹt giữa hai bức tường theo hai hướng liền kề nhau, nó cũng không thể di chuyển. Ví dụ, nếu viên đá bị chặn bởi một bức tường ở phía trên và một bức tường ở bên trái, thì trạng thái này cũng được xem là deadlock.



Hình 7: Trường hợp minh họa

Kiểm tra deadlock yêu cầu xét toàn bộ các viên đá. Nếu một trong số chúng rơi vào trạng thái không thể di chuyển, trạng thái đó bị xem là deadlock.

Độ phức tạp của thuật toán kiểm tra deadlock thường tỷ lệ thuận với số viên đá (n), tức là: $O(n)$. Mặc dù độ phức tạp khá nhỏ, lợi ích của việc loại bỏ các trạng thái deadlock lại rất lớn. Nó giúp tiết kiệm thời gian xử lý, giảm đáng kể số trạng thái cần xét, và tối ưu hóa bộ nhớ, từ đó cải thiện hiệu suất của thuật toán tìm kiếm.

5 Kết quả thực nghiệm

5.1 Kết thuật toán

5.1.1 Bản đồ 1

Test Case	Algorithm	Steps	Cost	Nodes	Time (ms)	Memory (MB)
[t] Input 1	BFS	36	110	441	25.48	0.18
	DFS	62	110	565	35.17	0.17
	UCS	36	110	737	37.38	0.19
	A*	36	110	177	10.14	0.07
	GBFS	38	110	84	5.04	0.04

Bảng 3: Kết quả thực thi các thuật toán của input 1

Test Case	Algorithm	Steps	Cost	Nodes	Time (ms)	Memory (MB)
[t] Input 2	BFS	62	190	5727	427.32	2.51
	DFS	1516	3250	6633	527.31	4.22
	UCS	62	190	9238	627.02	3.04
	A*	62	190	3146	272.25	0.96
	GBFS	100	270	888	70.28	0.27

Bảng 4: Kết quả thực thi các thuật toán của input 2

Test Case	Algorithm	Steps	Cost	Nodes	Time (ms)	Memory (MB)
[t] Input 3	BFS	67	320	95877	26579.12	42.49
	DFS	8784	19390	62980	18570.33	174.48
	UCS	67	320	205117	58065.34	70.14
	A*	67	320	51443	17622.70	18.37
	GBFS	80	320	1196	262.92	0.35

Bảng 5: Kết quả thực thi các thuật toán của input 3

5.1.2 Bản đồ 2

	Test Case	Algorithm	Steps	Cost	Nodes	Time (ms)	Memory (MB)
[t]	Input 4	BFS	43	90	889	127.60	0.38
		DFS	121	110	1982	170.08	0.66
		UCS	43	90	1417	95.37	0.44
		A*	43	90	584	49.73	0.17
		GBFS	43	90	316	22.00	0.12

Bảng 6: Kết quả thực thi các thuật toán của input 4

	Test Case	Algorithm	Steps	Cost	Nodes	Time (ms)	Memory (MB)
[t]	Input 5	BFS	58	260	56383	14269.70	22.45
		DFS	1045	1280	34893	6502.40	32.36
		UCS	59	190	62502	5843.55	20.02
		A*	59	190	11357	984.48	3.85
		GBFS	72	190	558	42.21	0.19

Bảng 7: Kết quả thực thi các thuật toán của input 5

	Test Case	Algorithm	Steps	Cost	Nodes	Time (ms)	Memory (MB)
[t]	Input 6	BFS	36	360	49107	6036.65	24.00
		DFS	1525	3480	1917	149.95	2.47
		UCS	42	300	247977	71248.13	80.04
		A*	42	300	17017	5802.33	5.91
		GBFS	84	520	1105	251.18	0.32

Bảng 8: Kết quả thực thi các thuật toán của input 6

5.1.3 Bản đồ 3

Test Case	Algorithm	Steps	Cost	Nodes	Time (ms)	Memory (MB)
Input 7	BFS	31	180	2049	346.02	0.61
	DFS	265	300	1403	272.62	0.56
	UCS	31	180	2329	370.72	0.64
	A*	31	180	965	192.54	0.30
	GBFS	35	180	279	48.90	0.10

Bảng 9: Kết quả thực thi các thuật toán của input 7

Test Case	Algorithm	Steps	Cost	Nodes	Time (ms)	Memory (MB)
Input 8	BFS	49	360	68887	18842.60	24.74
	DFS	3111	5400	19739	3337.29	15.50
	UCS	49	360	81041	20089.11	27.88
	A*	49	360	25671	11355.40	9.98
	GBFS	57	420	693	204.01	0.22

Bảng 10: Kết quả thực thi các thuật toán của input 8

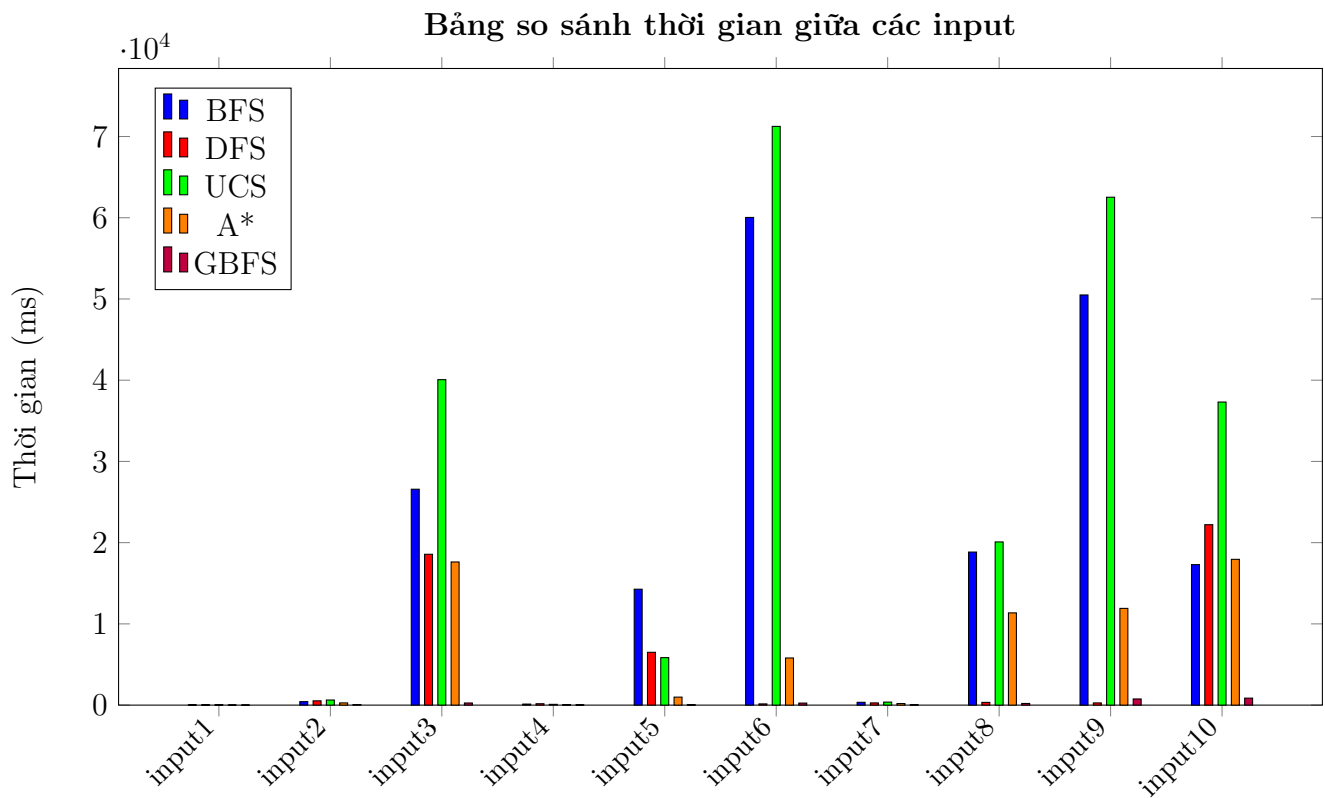
Test Case	Algorithm	Steps	Cost	Nodes	Time (ms)	Memory (MB)
Input 9	BFS	51	610	200851	50492.20	80.93
	DFS	2123	4130	7276	270.80	65.17
	UCS	55	450	94424	62526.31	65.79
	A*	55	450	60186	11908.48	24.41
	GBFS	93	770	7138	758.40	13.69

Bảng 11: Kết quả thực thi các thuật toán của input 9

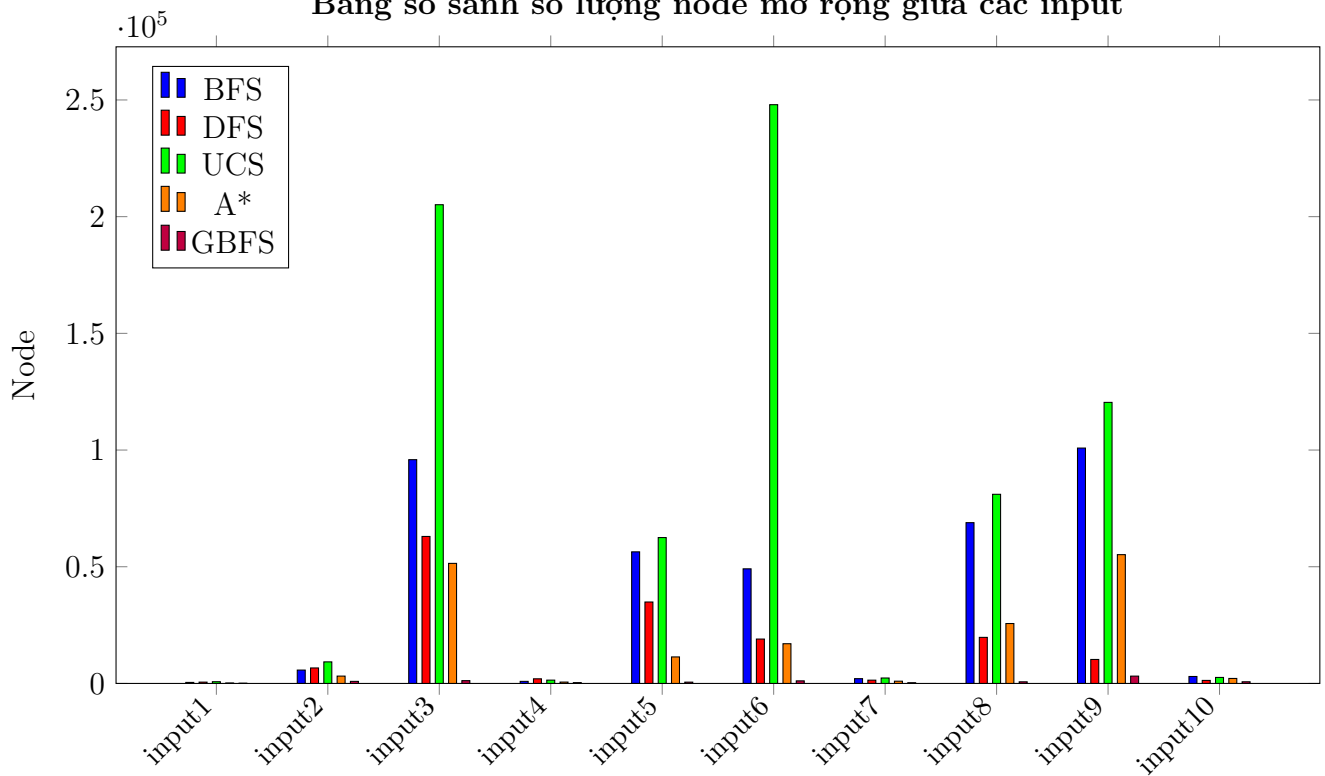
Test Case	Algorithm	Steps	Cost	Nodes	Time (ms)	Memory (MB)
Input 10	BFS	57	280	100086	17295.42	32.35
	DFS	5119	4940	61402	22210.79	36.24
	UCS	73	200	92774	37310.71	29.54
	A*	73	200	44197	17947.63	15.00
	GBFS	107	340	2609	859.89	0.85

Bảng 12: Kết quả thực thi các thuật toán của input 10

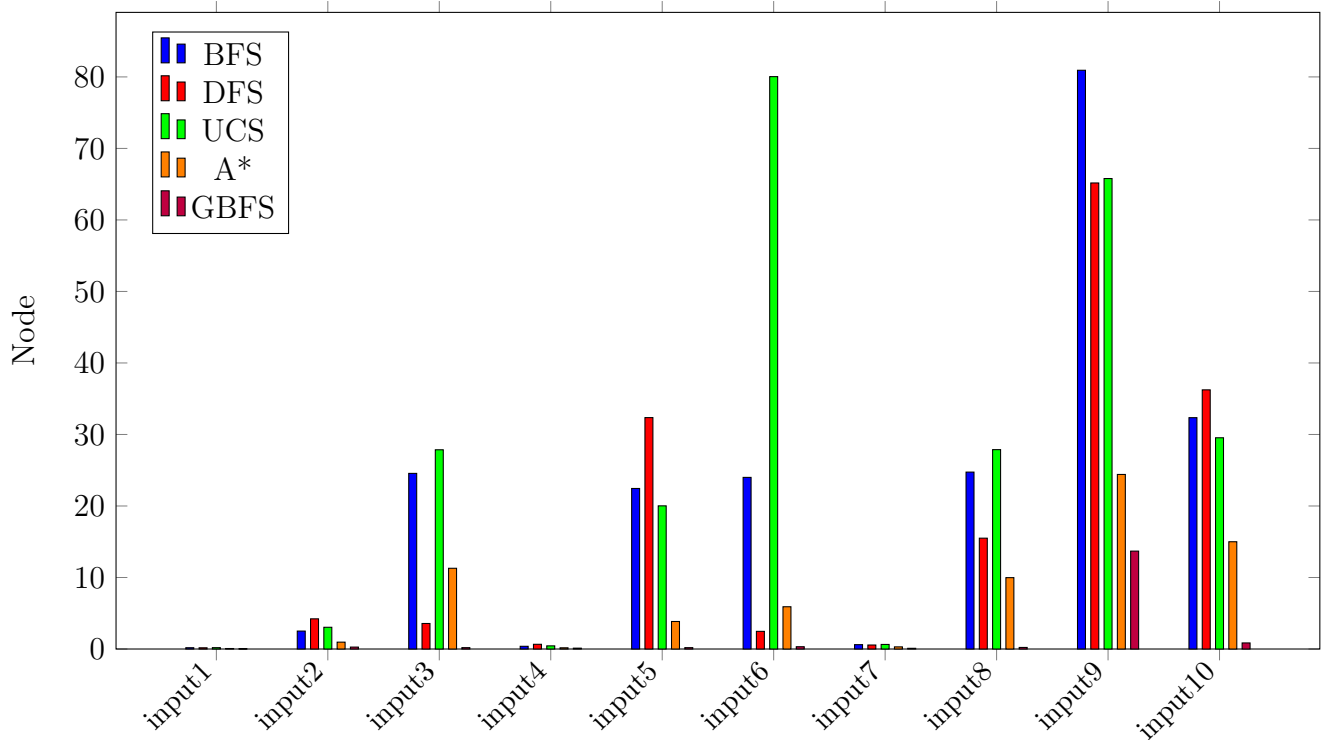
5.2 Biểu đồ



Bảng so sánh số lượng node mở rộng giữa các input



Bảng so sánh memory giữa các input



5.3 Phân tích test case

Nhìn vào bảng phân số liệu trên khi thực thi các input file do nhóm em tạo ra với nhiều tiêu chí, ở các mức độ khác nhau, chúng ta thấy được sự khác biệt giữa các thuật toán.

Đầu tiên, về yếu tố thời gian, thuật toán có tốc độ nhanh là **GBFS** với việc tận dụng thông tin, kế tiếp là nhóm tốc độ trung bình gồm **A*** và **DFS**, với **A*** cũng tận dụng việc có thông tin địa hình nhưng phải tìm ra kết quả tối ưu nhất nên tốn nhiều thời gian hơn. **DFS** là thuật toán duyệt theo chiều sâu, thường dựa vào yếu tố may mắn nếu kết quả ở hướng mà DFS đang duyệt, còn không sẽ tốn thời gian rất lâu, nhưng thời gian trung bình vẫn ở mức chấp nhận được. Cuối cùng, nhóm có thời gian lâu nhất là **BFS** và **UCS**, với **BFS** duyệt theo chiều rộng, nên nếu kết quả là một chuỗi bước dài sẽ dẫn đến việc số lượng node được mở lên tăng theo hàm mũ. **UCS** tối ưu hóa đường đi nhưng gặp phải vấn đề về cách chọn trạng thái có giá trị nhỏ nhất, nên sẽ tránh phải đẩy các viên đá, đến khi không còn cách nào khác mới đẩy khiến cho tốn thời gian duyệt qua các trạng thái không cần thiết.

Tiếp theo, về yếu tố bộ nhớ và số lượng trạng thái node mở rộng, **GBFS** vẫn sử dụng rất ít khi không cần phải mở rộng và lưu trữ quá nhiều trạng thái trong hàng chờ, trong khi các thuật toán duyệt theo chiều rộng như **BFS** và **UCS** sẽ tiêu tốn nhiều vào phần lưu trữ trạng thái trong hàng chờ, nhất là với **UCS** khi cần phải tìm ra đường đi tốt nhất. Cuối cùng là **DFS** với việc phụ thuộc vào hướng đi có đúng hay không, phụ thuộc vào yếu tố may mắn, nếu không cũng sẽ mở và lưu trữ rất nhiều trạng thái để duyệt. **A*** nhờ có thông tin và cũng tiêu tốn cho việc tìm kiếm tối ưu nên sử dụng bộ nhớ ở mức trung bình.

Qua các mức độ của input file, ta có thể thấy được tùy thuộc vào số lượng đá và kích thước của mê cung, thời gian và bộ nhớ tiêu thụ sẽ khác nhau, với các test như input 1, input 4 và input 7 chỉ có 1 viên đá thì thời gian chạy rất ngắn với hầu hết thuật toán, điều này dễ hiểu thông qua việc chỉ cần đưa 1 viên đá đến vị trí mục tiêu nên sẽ dễ dàng hơn. Với các input 2, input 5, input 8 với số lượng đá là 2 cũng chưa quá đáng kể, chỉ nhiều hơn một chút về số lượng node mở rộng qua việc đẩy 2 viên đá về vị trí đúng và với UCS và A-star thì tối ưu về khối lượng đẩy và các số bước di chuyển. Đến với các input khó nhất là input 3, input 6, input 9 và input 10 sẽ mang đến sự khác biệt lớn giữa các thuật toán qua việc đẩy cả 3 viên đá về vị trí đúng, số lượng node lúc năng tăng lên nhiều hơn do nhiều trạng thái có thể xảy ra hơn trước khi đến với trạng thái đích, ngoài ra cũng tạo vấn đề cho các thuật toán tối ưu như UCS và A-star.

6 Kết Luận

Dựa trên kết quả đánh giá nhiều lần thực nghiệm của nhóm để giải quyết bài toán Ares'Adventure nhóm chúng em đã tổng hợp những thuận lợi và khó khăn của từng thuật toán trong việc giúp Ares hoàn thành nhiệm vụ của mình như sau.

6.1 Ưu điểm

6.1.1 BFS

Thuật toán tìm kiếm theo chiều rộng (BFS) nổi bật nhờ tính đơn giản và đảm bảo tìm ra lời giải với số bước di chuyển ít nhất khi mọi thao tác có cùng chi phí. Việc khám phá các nút theo từng cấp độ giúp đảm bảo rằng giải pháp được tìm thấy ở độ sâu tối thiểu, đặc biệt hữu ích trong các tình huống mà số bước đi quan trọng hơn tối ưu hóa chi phí. Ngoài ra, BFS hoạt động hiệu quả

trên các bản đồ nhỏ hơn do không yêu cầu bộ nhớ bổ sung để lưu trữ cấu trúc dữ liệu phức tạp hoặc thực hiện các tính toán heuristic.

6.1.2 DFS

Tìm kiếm theo chiều sâu (DFS) đặc biệt hữu ích trên những bản đồ hẹp hoặc có ít nhánh rẽ. Nhờ khả năng tiêu thụ bộ nhớ thấp, DFS trở thành lựa chọn tốt trong môi trường có giới hạn tài nguyên. Bên cạnh đó, tốc độ tìm kiếm nhanh trong không gian hẹp giúp DFS đôi khi được sử dụng như một bước kiểm tra sơ bộ để đánh giá tính khả thi của lời giải. Khi kết hợp với các tối ưu hóa nhằm tránh vòng lặp vô hạn, DFS có thể nhanh chóng khám phá các đường đi sâu, khiến nó phù hợp cho một số nhiệm vụ tìm kiếm thử nghiệm.

6.1.3 UCS

Thuật toán Uniform Cost Search (UCS) đảm bảo tìm ra giải pháp tối ưu, tức là đường đi có chi phí thấp nhất từ điểm bắt đầu đến điểm đích, nếu tồn tại. Nó có thể áp dụng cho mọi loại đồ thị, kể cả đồ thị có chi phí khác nhau giữa các cạnh, miễn là các chi phí luôn không âm. Bên cạnh đó, UCS sử dụng hàng đợi ưu tiên (Priority Queue) để tìm kiếm theo chi phí tối thiểu một cách hiệu quả, với độ phức tạp phụ thuộc vào kích thước của đồ thị và cấu trúc dữ liệu được sử dụng.

6.1.4 A*

Thuật toán A* sử dụng chi phí thực tế kết hợp với kết quả chi phí dự đoán heuristic điều này giúp cho thuật toán sẽ tìm được kết quả tối ưu nhất nhờ việc xác định chi phí thực tế giúp cho việc tìm kiếm không bị lan man, ngoài ra còn cải thiện tốc độ xử lý bởi việc áp dụng hàm ước lượng heuristic.

6.1.5 GBFS

Greedy Breadth First Search (GBFS) có tốc độ nhanh vì ưu tiên các trạng thái có ước lượng tốt nhất (gần mục tiêu nhất), giúp giảm số lượng trạng thái phải duyệt so với BFS và DFS. Vì chỉ xét đến heuristic, thuật toán thường tìm ra lời giải nhanh, đặc biệt hữu ích trong các bài toán thời gian thực. Việc chỉ xét đến heuristic còn giúp tiết kiệm tài nguyên hơn so với UCS và A*.

6.2 Nhược điểm

6.2.1 BFS

Bởi vì phải duyệt qua tất cả trạng thái và lưu trong hàng đợi, BFS chỉ duyệt qua những trạng thái lân cận nên có thể dẫn đến tốn kém bộ nhớ, tốn nhiều thời gian thực thi. Khi số lượng trạng thái mở rộng quá lớn, đặc biệt với bản đồ có kích thước lớn hoặc nhiều chướng ngại vật, BFS có thể gặp vấn đề nghiêm trọng về hiệu suất. Ngoài ra, vì BFS mở rộng theo từng lớp nên nếu không có điều kiện cắt nhánh hợp lý, thuật toán có thể phải kiểm tra rất nhiều trạng thái dư thừa, làm tăng đáng kể độ phức tạp tính toán.

6.2.2 DFS

Nếu không kiểm tra trạng thái đã duyệt, thuật toán DFS có thể rơi vào vòng lặp vô hạn, đặc biệt trong đồ thị có chu trình. Hơn nữa, DFS không đảm bảo tìm ra lời giải tối ưu, vì nó có thể đi theo một nhánh sâu trước khi khám phá các nhánh khác, dẫn đến việc bỏ lỡ đường đi ngắn nhất. Khi không gian trạng thái có độ sâu quá lớn, việc triển khai DFS bằng đệ quy có thể gây lỗi tràn ngăn xếp (stack overflow).

6.2.3 UCS

Thuật toán Uniform Cost Search (UCS) có thể yêu cầu rất nhiều bộ nhớ, đặc biệt đối với các đồ thị lớn, do phải lưu trữ tất cả các trạng thái đã duyệt cũng như các trạng thái còn lại trong hàng đợi ưu tiên. Khi áp dụng cho đồ thị có nhiều trạng thái hoặc chi phí cao, UCS có thể mất nhiều thời gian để tìm kiếm, dẫn đến hiệu suất kém hơn so với một số thuật toán khác như BFS hoặc DFS trong một số trường hợp.

6.2.4 A*

Thuật toán A-star phụ thuộc vào hàm heuristic, nên nếu hàm heuristic không đáng tin cậy sẽ khiến cho việc xử lý không được tối ưu như kỳ vọng hoặc tệ hơn là dẫn đến kết quả sai lệch. Ngoài ra, việc phải tìm ra đường đi tối ưu cũng mắc một khuyết điểm tương đồng với UCS là tiêu hao bộ nhớ khá nhiều để xem xét hết các trạng thái trong hàng đợi cho đến khi lấy ra được trạng thái đích.

6.2.5 GBFS

GBFS không đảm bảo tìm được đường đi tối ưu do chỉ quan tâm đến khoảng cách ước lượng mà không xét chi phí thực tế. Nếu hàm heuristic không tốt, thuật toán có thể đi vào hướng sai lầm và bỏ lỡ con đường tốt hơn. Trong một số trường hợp, nếu trạng thái bị heuristic đánh giá sai, thuật toán có thể không tìm được lời giải dù có tồn tại.

7 Biến thể thuật toán A* (Weighted A*)

Trong quá trình áp dụng thuật toán A* để giải quyết bài toán *Ares' Adventure*, nhóm em nhận thấy rằng việc thay đổi hệ số trọng số w trong hàm heuristic dẫn đến kết quả đầu ra khác nhau. Ta định nghĩa công thức sau:

$$f(n) = g(n) + W \cdot h(n) \quad (1)$$

Trong đó:

- $g(n)$ là chi phí thực tế từ trạng thái ban đầu đến trạng thái n .
- $h(n)$ là giá trị heuristic ước lượng chi phí từ n đến trạng thái đích.
- W là hệ số trọng số, điều chỉnh mức độ ưu tiên của heuristic so với chi phí thực tế.

Algorithm	Steps	Weight	Nodes	Time (ms)	Memory (MB)
A*	125	430	16832	3475.22	5.23
Weighted A* (w=0.5)	125	430	17564	3489.55	5.20
Weighted A* (w=5)	153	470	12909	2700.92	4.10
Weighted A* (w=50)	211	730	5126	1018.14	1.98

Bảng 13: Kết quả các thuật toán trên input 14

Algorithm	Steps	Weight	Nodes	Time (ms)	Memory (MB)
A*	193	1050	343759	91508.07	111.82
Weighted A* (w=0.5)	193	1050	347126	96006.44	111.39
Weighted A* (w=5)	259	1250	278031	72881.06	88.31
Weighted A* (w=50)	327	1590	26972	7710.20	10.20

Bảng 14: Kết quả thuật toán trên input 15

Algorithm	Steps	Weight	Nodes	Time (ms)	Memory (MB)
A*	25	64	2462	390.90	0.99
Weighted A* (w=0.5)	25	64	4792	902.17	1.76
Weighted A* (w=5)	40	64	1912	270.79	0.67
Weighted A* (w=50)	49	64	406	62.34	0.15

Bảng 15: Kết quả các thuật toán trên input 16

Qua kết quả tổng hợp từ các input ta nhận thấy rằng với:

- Hệ số $0 < W < 1$ làm giảm ảnh hưởng của giá trị dự đoán của heuristic trong $f(n)$, giúp cho thuật toán tìm được đường đi tối ưu hơn, điều này đồng nghĩa với việc thuật toán phải duyệt qua nhiều node hơn để tìm kiếm đường đi tối ưu nhất.
- Hệ số $W > 1$ làm tăng ảnh hưởng của giá trị dự đoán của heuristic trong $f(n)$, quá đó giúp thuật toán tìm được đường đi với thời gian nhanh chóng hơn. Tuy nhiên nếu với W quá lớn (Như trong ví dụ với $W = 50$) heuristic được ưu tiên quá mức dẫn đến việc đường đi được chọn không được tối ưu.

Nhu vậy ta thấy được qua việc lựa chọn W phù hợp có thể giúp chúng ta tối ưu hóa mục đích của mình trong giải quyết các vấn đề đặt ra của bài toán. Đối với những bài toán yêu cầu tìm đường đi ngắn nhất ta sẽ ưu tiên sử dụng W nhỏ. Trong khi đó đối với bài toán yêu cầu về tốc độ cao nhanh chóng đưa ra đường đi thì ta nên cân nhắc sử dụng hệ số W lớn và phải chúng nhận đường đi sẽ không được tối ưu.

8 Tham khảo

- *CS50's Introduction to Artificial Intelligence with Python CS50's Introduction to Artificial Intelligence with Python*
- *Artificial Intelligence A Modern Approach Book*
- *Pathfinding Visualizer*
- *A* And Weighted A* search Trường Đại học Maxim Likhachev Carnegie Mellon*